Summary Notes:

DevOps Basics and Introduction to Docker

1. Introduction

Overview

Here is an overview of the topics covered in the session:

1. DevOps Basics
    - Minimum Viable Product
    - Software Development Life Cycle
    - Waterfall Model
    - Agile Model
    - Agile Scrum
    - DevOps
    - Introduction to CI/CD pipeline
    - Principles of DevOps
    - DevOps practices
2. Version Control Systems
    - Version control
    - Centralised Version Control Systems
    - Distributed Version Control Systems
    - Git and various commands
    - GitHub
3. Introduction to Docker
    - Docker
    - Docker v/s Virtual Machines
    - Docker Architecture
    - Docker Image
    - Docker Registry
    - Docker containers
    - Docker volume

© Copyright upGrad Education Pvt. Ltd. All rights reserved

2. DevOps Basics

2.1 Minimum Viable Product

Minimum Viable Product (MVP) is a term in product development that refers to the first version of a product that has only enough features to satisfy early adopters and gather feedback for future development. It should be built quickly with minimum resources and effort required to prove out a product idea or hypothesis.

Important Features of an MVP:

- Core functionality: In MVP, only the most significant aspects which address the main issue or demand of target users are included. Emphasis is on value delivery through minimal complexity.
- User Feedback: This involves releasing an MVP for companies so as to get insights from real end-users. This helps in understanding what users want, their preferences and possible problems with the product.
- Speed and Efficiency: The strategy for building an MVP aims at getting a product released as soon as possible. This helps teams avoid spending too much time and money on features that might not be necessary or wanted by users.
- Iterative Development: After an MVP is launched, it can be expanded and improved based on user feedback this means that the product becomes iterative. Through this process, companies make continuous improvement leading to better alignment with market expectations.
- Risk Reduction: By creating an MVP, the risk of launching an unmarketable product is mitigated. It allows for hypothesis testing and data-driven decisions before going big.

For example, a start up might want to develop a new kind of fitness app that can be used on mobile phones. Instead of creating fully-functional software with nutritional advice, social media support and tailored exercise routines, the company could release an MVP with only basic workout monitoring. Such a launching can help them find out if people are interested in it as well as what features they would like to see most.

© Copyright upGrad Education Pvt. Ltd. All rights reserved

To summarize, MVP is an operational methodology for developing products that prioritizes user feedback, velocity and incremental progress. Companies can employ it to tryout ideas, listening to customers' opinions and adapting rapidly to the changing market conditions.

2.2 Software Development Life Cycle

The process of creating software applications is called Software Development Life Cycle (SDLC), this continuous process encompasses several steps that provide a framework for organizing, building, testing and implementing information systems.

SDLC's goal is to ensure that software is produced in line with customer's expectations, on time, within budget and compatible with the current and future IT infrastructure.

Key stages of the SDLC:

1. Requirements Analysis & Planning
    - Objective of this project is to understand its scope and collect requirements from relevant stakeholders.
    - Actions include cost estimation, resource allocation, risk assessment and feasibility studies.
    - Result: comprehensive document that outlines the requirements
2. System Design:
    - The foregoing procedure aims at translating the collected requirements into a detailed system architecture and design.
    - Activities: Determining architecture, interfaces, system components; Producing both high-level and detailed design documents.
    - Result: System design documentation that can guide programmers.
3. Coding (implementation):
    - Purpose here is writing the program code based on those design documents.
    - Actions such as developers writing codes, setting up development environments as well as integrating different parts of a system are taken by developers.
    - Final Product: source code for the software.
4. Testing

© Copyright upGrad Education Pvt. Ltd. All rights reserved

- The objective here is to detect and correcting defects in software. Some activities performed involve carrying out user acceptance tests, unit tests, integration testing plus system testing.
- Result: A software product that is reliable and works well, with test cases and findings that are recorded.

5. Deployment:
    - To deploy the program in the live environment for users' access.
    - Activities: Production environment configurations, live server deployment of the program and system setups completion.
    - Final Product: A software program that works and is active.
6. Maintenance & Support:
    - Objective: To continue supporting and maintaining the software after it has been deployed.
    - Activities: Addressing bugs, upgrading, boosting speed, and modifying it based on user feedbacks.
    - Outcome: Regular improvement of the programme to ensure that it remains useful and functional.

2.3 Waterfall Model

The Waterfall Model is a traditional sequential approach to developing software which divides the process into several distinct stages. There is little overlap between these stages; each stage must be completed before moving to the next. The name "Waterfall Model" refers to this because like a waterfall it moves in one direction from top to bottom only.

Key Stages in the Waterfall Model

1. Requirements Collection and Analysis
    - Goal: Acquire all functional and non-functional requirements from the stakeholders and document them.
    - Activities: Detailed requirements gathering through meetings, questionnaires, interviews.
    - Output: A detailed requirement specification paper outlining software functions that are required.
2. System Designing:

© Copyright upGrad Education Pvt. Ltd. All rights reserved

- Goal: Use collected requirements to build an architecture and design for a system.
- Activities: Creating full and high level designs incorporating interface specifications, data models, and system architecture.
- Output: Design documents functioning as development blueprints.

3. Coding (Implementation):
   - Objective: Writing code based on the design specifications to create the actual software.
   - Activities: It involves integrating various modules, configuring development environments, coding,
   - Result: Software's source code.
4. Integration & Testing:
   - Objective : To verify that the program satisfies specifications and works as planned
   - Activities : User Acceptance Testing , System testing , Integration testing , Unit testing
   - Outcome : A fully tested software package with problems found & addressed
5. Deployment
   - Objective: Execute the program in a production environment where end-users will be able to use it.
   - Activities: Last minute system checks, user training and putting the program on production servers.
   - Result: The program comes online and is ready for use.
6. Maintenance
   - Objective: Deliver uninterrupted maintenance and support to cater for any issues or adjustments that occur after deployment.
   - Activities: fixing bugs, updating software, adding new features based on users' suggestions.
   - Result: The software shall remain relevant and functional with time.

Advantages of the waterfall model

© Copyright upGrad Education Pvt. Ltd. All rights reserved

- Simple and Easy to Understand – It has a linear structure which makes it very easy to follow and manage.
- Clear Documentation – Easy tracking of progress and understanding what state of the project has been achieved when each stage has distinct deliverables and documentation.
- Clearly stated Requirements- The project begins with an all-inclusive requirement analysis culminating into clearly stated objectives.

Waterfall Model Drawbacks

- Substantial rigidity: it becomes difficult to modify or adjust something when the phase has ended. This can be a problem when there is a change in specifications.
- Late testing; this is known as late testing and it occurs after the implementation stage which may result in finding problems farther down the process and needs much more money and effort to solve.
- Not good for Complex and Large Projects: this assumption is that all requirements can be completely specified upfront however, that's not always true.

2.4 Agile Model

The **Agile Model** is a flexible, iterative approach to software development that stresses collaboration, customer feedback, and fast delivery of small incremental changes. Unlike Waterfall Model, this model allows for an ongoing evolution/adaptation throughout the entire development process.

Key Principles of Agile Model:

1. **Iterative Development:**
    - Agile initiatives are often divided into small manageable units called iterations or sprints lasting two to four weeks. Each iteration covers planning, developing, testing and reviewing.
2. **Customer Collaboration:**

© Copyright upGrad Education Pvt. Ltd. All rights reserved

- Throughout the development process, customers and stakeholders contribute actively by continuously providing feedbacks and explanations. This helps to ensure that the final product matches closely with users' needs and expectations.

3. **Incremental Delivery**
    - The software is developed and delivered in small functional increments instead of one monolithic release. Users can then experience some part of it early and often.
4. **Flexibility and Adaptability:**
    - Agile accepts adjustments thus allowing teams to respond to new demands or shift priorities even late in the project cycle. Through constant planning as well as review, this adaptability is made possible.
5. **Cross-functional Teams:**
    - Agile teams are commonly cross-functional that implies they have members having different skills for undertaking the project (such as developers, testers, designers). This cultivates collaboration as well as quick solving of problems.
6. **Continuous Improvement:**
    - At the end of each iteration, teams think about the things that went well and what could be done better. Often, these reflections are formalized through a retrospective meeting which will lead to continuous growth in processes.

Common Agile Frameworks:

1. **Scrum:**
    - Scrum is one of the most popular Agile frameworks. It splits work into sprints and includes roles such as Product Owner, Scrum Master and Development Team. Daily scrum meetings, sprint reviews and retrospectives are key aspects of Scrum.
2. **Kanban:**
    - Kanban mainly focuses on visualizing workflow and managing work in progress. A Kanban board with columns for different stages of work is used so that teams can quickly assess task statuses.
3. **Extreme Programming (XP):**

© Copyright upGrad Education Pvt. Ltd. All rights reserved

- Technical excellence and customer satisfaction are emphasized in XP. For example, it encourages practices such as pair programming, test-driven development (TDD) and continuous integration.

4. **Lean Software Development:**
    - Lean concentrates on cutting down waste while increasing value. Its aim is to deliver software efficiently by identifying any redundancy during the process or activities involved.

Advantages of the Agile Model:

- **Flexibility and Adaptability:** Agile is amenable to requirements change even after a project has begun, thereby making it useful in projects with mutable needs.
- **Customer-Centric:** Constant customer and stakeholder involvement ensures that the final product matches their needs and expectations.
- **Early and Continuous Delivery:** Frequent working software releases enable early feedback from users, leading to quick delivery of value.
- **Improved Quality:** Regular testing and reviews lead to early detection of defects, resulting in better quality.

Disadvantages of the Agile Model:

- **Scope Creep:** Sometimes, agility may introduce scope creep as changes are not well managed.
- **Less Predictability:** Determining the end cost, time frame or total work load for an iteration can be quite challenging at inception.
- **Requires Skilled Teams:** It is difficult for these organizations with less experienced staffs when it comes to handling agile that requires high level of competency and collaboration among team members.

Agile model fits perfectly into projects where changes are anticipated over time or unclear. This approach encourages an adaptive environment that encourages collaboration between teams hence allowing fast production of quality software products.

© Copyright upGrad Education Pvt. Ltd. All rights reserved

2.5 Agile Scrum

Agile Scrum is a popular Agile methodology, widely used in managing and completing intricate projects. It emphasizes on iterative and incremental product delivery thus enabling teams to adapt quickly to changes and improve continuously. The main focus of scrum is on specific roles, events and artifacts that allow for collective and efficient work operations.

Key Components of Scrum:

1. **Roles:**
    - Product Owner:
        - Stakeholders' representatives.
        - Define and prioritize the product backlog ensuring team delivers value.
        - It communicates the vision as well as goals for the product to the scrum team.
    - Scrum Master:
        - Acts as a facilitator or coach for the team.
        - Makes sure that the team adheres to scrum principles and practices.
        - Removes any obstacles that could hinder progress of the team.
        - Guards against interference from people outside the team.
    - Development Team:
        - A cross-functional group responsible for delivering increments of the product
        - Normally includes developers, testers, designers etc
        - Self-organized in deciding how to do their work within a sprint.
2. **Events (Ceremonies):**
    - Sprint:
        - A time-boxed period, usually of one to four weeks, in which the potentially shippable product increment is derived.
        - Each sprint starts with a planning and ends with a review and retrospective.
    - Sprint Planning:

© Copyright upGrad Education Pvt. Ltd. All rights reserved

        - A meeting where the Scrum team plans what work will be done during that sprint.
        - The team selects items from the product backlog and defines a sprint goal.
    - Daily Scrum (Stand-up):
        - A brief daily meeting (usually for 15 minutes) where teammates talk about their progress, day plan and impediments if any.
        - Helps keep the team synchronised and on track.
    - Sprint Review:
        - Held at the end of the Sprint to inspect completed work and gather feedback.
        - The team demonstrates increments to stakeholders; If necessary, update product backlog
    - Sprint Retrospective:
        - A meeting for the Scrum team to reflect on their last sprint and come up with ways to improve processes as well as teamwork.
        - Focuses on what went well, what didn't, how to fix it.
3. Artifacts:
    - Product Backlog:
        - A priority queue of all the work in the project that needs to be done and which is maintained by the Product Owner.
        - Comprises features, bug fixes, technology improvements and knowledge acquisition.
    - Sprint Backlog:
        - A subset of the product backlog selected for the sprint with a plan on how to deliver the increment.
        - Involves user stories and tasks that should be done during a sprint.
    - Increment:
        - The result of all items in product backlog completed during a sprint plus those from previous sprints.
        - This is meant to be potentially shippable product at every end of sprint.
    - Definition of Done:

© Copyright upGrad Education Pvt. Ltd. All rights reserved

- A shared understanding within the Scrum team about when work is considered complete
- It ensures consistency as well as quality of increments delivered.

Advantages of Scrum:

- **Flexibility and Adaptability:** Requirements changes can occur even late in development stage, thus responding to new information while meeting customers' demand.
- **Increased Transparency:** Regular meetings with clear roles promote communication and visibility for everyone's alignment purposes.
- **Continuous Feedback:** Prompt reviews together with stakeholders allow for early and continuous feedback towards a better product outcome.
- **Improved Quality:** By focusing on iterative development along with regular retrospectives, teams constantly enhance their processes and product quality.

Challenges and Considerations:

1. **Discipline Required:** Adhering to its principles and events can be a challenge for some teams using Scrum.
2. **Depends on Team Collaboration:** Effective communication and collaboration between team members are critical in the success of Scrum.
3. **Not Suitable for All Projects:** Scrum may not add much value to projects with well-defined, unchanging requirements.

Scrum is widely used in software development and has been adapted to various industries and types of projects. It is a useful framework that promotes quality product delivery through collaboration, flexibility, and continuous improvement in fast-changing conditions.

2.6 DevOps

DevOps as a movement advocates practices that aim at bridging the gap between software development (Dev) and IT operations (Ops). The main goal of DevOps is to improve

© Copyright upGrad Education Pvt. Ltd. All rights reserved

collaboration and communication between developers and operation teams so they can deliver faster, more reliable, and more efficient software products or services.

Key Concepts and Principles of DevOps:

1. Collaboration:
    - It encourages close interaction between developers, operators, other stakeholders who partake in the process of delivering software solutions.
    - This ensures that goals are aligned and there is a common understanding of project objectives.
2. Continuous Integration (CI):
    - Regularly merging code changes into a shared repository is what Continuous Integration entails. This enables software to always be in a deployable state by facilitating early issue detection through automated builds and tests.
3. Continuous Delivery (CD):
    - By automating the deployment process, Continuous Delivery expands on CI, making sure that software can be released to production at any time. To do this, there must be substantial testing of the functionality of the software in different environments using automation frameworks.
4. Infrastructure as Code (IaC):
    - Alternatively known as IaC, it refers to managing and provisioning computing infrastructure programmatically through machine readable scripts or configuration files instead of physical hardware configuration or interactive configuration tools. This permits version control, automated testing and continuous deployment environments.
5. Automation:
    - Such tasks as testing, deployment and infrastructure provisioning need to be automated for DevOps principle which aims at reducing manual efforts and errors. It encompasses repetitive task automation like software testing, infrastructure provisioning, deployment and monitoring.
6. Monitoring and Logging:

© Copyright upGrad Education Pvt. Ltd. All rights reserved

- With DevOps, applications and infrastructure must always be monitored for system reliability, performance and security purposes. Teams can quickly identify system behavior issues through logging thereby resolving them speedily.

7. Security (DevSecOps):
    - The term DevSecOps is often used to describe the practice of integrating security practices into the development and operational processes. This ensures that security concerns are addressed from inception, throughout the software life cycle.

Benefits of DevOps:

1. Reduction in Time to Market:
    - DevOps facilitates faster development cycles and more rapid deployment of new features and updates through automation and improved collaboration.
2. Enhanced Quality and Reliability:
    - Constant testing and monitoring enable early identification of issues leading to more stable and dependable software releases.
3. Increased Efficiency:
    - Automation mitigates manual interventions thereby freeing resources as well as minimizing errors made by human beings.
4. Scalability and Flexibility:
    - In response to changing needs, teams can quickly scale applications as well as infrastructure using Infrastructure as Code with automated deployment processes.

Challenges of DevOps:

1. **Cultural Change**:
    - Transitioning to a DevOps culture requires a shift in mindset and practices, which can be challenging for organisations with entrenched processes and silos.
2. **Complexity:**
    - Ingrating different tools and processes can be complicated which calls for skilled professionals as well as good management.
3. **Security Concerns:**

© Copyright upGrad Education Pvt. Ltd. All rights reserved

- Nevertheless, rapid deployment cycles and extensive automation that are associated with DevOps can at times overlook security when not managed effectively.

4. **Tooling and Integration:**
    - However, the DevOps ecosystem comes with a vast array of tools; therefore, it is difficult to choose the right ones and ensure their smooth integration.

2.7 Introduction to CI/CD Pipeline

A Continuous Integration / Continuous Delivery (CI/CD) pipeline is a system of automated methods supported by instruments that enable software development teams to incessantly integrate, test, deliver and deploy applications. In other words, CI/CD means Continuous Integration / Continuous Deployment practices which aim to enhance software quality while speeding up the release cycle.

Key Concepts in CI/CD:

1. **Continuous Integration (CI):**
    - This refers to a practice that involves frequently merging code changes from multiple contributors into shared repository. It mainly aims at identifying early integration issues thereby improving coding integrity as well as reducing the time taken to attain finished software.
    - **Key Activities:**
        - **Automated Builds:** Every time code is committed, a process of automated build begins to compile the code and create executable objects.
        - **Automated Testing:** Code functionalities are validated through execution of automated tests (i.e. unit testing, integration testing…etc.)
2. **Continuous Delivery (CD):**
    - It is through Continuous Delivery that CI streamlines the process of moving code changes into a staging or production environment once the build and test stages have been successful. The important thing here is to make sure that the codebase can be deployed at any time.
    - **Key Activities:**

© Copyright upGrad Education Pvt. Ltd. All rights reserved

- **Deployment Automation:** Alterations in codes are automatically moved to testing platforms or even into staged environments.
- **Manual Approval Gates:** However, final deployment to production might need manual approval checks for conformance and security.

3. **Continuous Deployment (CD):**
    - Continuous Deployment takes Continuous Delivery a step further by automatically deploying every single change made on code that has passed all stages of the pipeline into production. This enables speedy frequent release cycles.
    - **Key Activities:**
        - **Full Automation:** Any change which passes automated tests has no manual approval gates before it's deployed directly into production.

2.8 Principles of DevOps

Gene Kim popularised "The Three Ways" in DevOps, which are basic concepts for implementing DevOps in an organisation.

1. Systems Thinking

Systems Thinking gives more focus to understanding the performance of the entire system and not individual components. In this view, teams look at how different parts of the organization, processes and technologies interact and influence each other holistically. The systems thinking aspect is manifested in:

- **End-to-End View:** Considering all parts of the system from development and testing to deployment and operations ensuring concerted efforts towards a single goal.
- **Removing Bottlenecks:** Identifying constraints or bottlenecks that hinder work flowing within system such as slow deployment process or communication silos.
- **Shared Responsibility:** Promoting collaboration between traditionally separate roles (e.g., development, QA, operations), encouraging shared responsibility for overall success of a system.

2. Amplifying Feedback Loops

© Copyright upGrad Education Pvt. Ltd. All rights reserved

Amplifying Feedback Loops entails developing mechanisms for quick and frequent feedbacks at each stage of development and operation processes with the aim of early problem detection and constant improvement. Key aspects include:

- **Fast Feedback:** This means identifying and communicating problems quickly in order to address them promptly like when the code is broken or tests fail. This approach helps realize time saved on fixing bugs as well as less impact on the system.
- **Continuous monitoring:** It also involves deployment of systems that monitor and give out real-time alerts concerning application performance, security, user experience among other areas. This ensures efficient incident response by teams.
- **Iterative Improvement:** Feedback loops are used to continuously improve the codebase, infrastructure, and processes. There are regular reviews, retrospectives, refinements happening.

3. Culture of Continual Experimentation and Learning

- **Culture of Continual Experimentation and Learning** encourages innovation, risk-taking culture, where failure is treated as a learning process. This principle emphasises:
- **Experimentation:** The approach includes experimenting with new technologies like deployment strategies or organizational structure through which teams work together to establish alternative ways of delivering value.
- **Learning from Failures:** Then this further demonstrates that failures are not seen negatively but can be transformed into lessons**.**
- **Encouraging Innovation**: Creating a safe environment that encourages innovation where members of the team feel empowered to put forth and try new ideas without being afraid of negative repercussions. This can result in breakthroughs and continuous improvement.

2.9 DevOps practices

In the modern software development landscape, **DevOps practices** are critical in bridging the gap between development and operations teams, streamlining workflows, and enabling continuous delivery and deployment. These practices involve several methodologies as well as tools for proficiently developing, deploying, and managing software. These include some core

© Copyright upGrad Education Pvt. Ltd. All rights reserved

DevOps practices such as Version Control, containerisation as well as Infrastructure as Code (IaaC) that are essential to a successful DevOps implementation.

1. Version Control

Version control is a basic practice in DevOps which involves tracking and managing changes made to software codes. It allows several developers to collaborate on the same code base swiftly but safely by providing ways to track changes or merge different branches of code or even go back to how things were before any modifications were made if necessary.

- **Tracking Changes:** VCS systems like Git keep records of modifications made to files in a repository. Each change is documented as a “commit,” together with extra data like author and timestamp, enabling teams to retrace the course of a project.
- **Branching and Merging:** Various developers can work on different branches of the codebase thus making it easier for them to add new features or fix bugs without affecting the main software. After completion, these branches are often merged back into their main branch possibly following an automated testing phase and code review.
- **Conflict Resolution:** Conflicts arise when several programmers modify the same file or line of code. Version control systems have conflict identification and resolution tools so that different contributions could be smoothly integrated.
- **Reversion:** In case an updated modification causes bugs or problems, version control helps programmers return back to previous stable state. This feature greatly cuts downtime leading to better quality software maintenance.

2. containerisation

Containerization is the practice of putting an application and its dependencies into a “container.” This package is stand-alone, light-weight, and can be executed. It contains everything that needs to run the application such as runtime, libraries and environment variables.

- **Consistency across environments:** Containers ensure consistent operation of applications in different environments from development to production. Since all required

© Copyright upGrad Education Pvt. Ltd. All rights reserved

configurations and dependencies are bundled within the container, there will be no more instances of 'It works on my machine'.

- **Isolation:** Containers can isolate applications to prevent interference among them. This is very important in microservices architectures where several services can run on the same host.
- **Scalability:** Containers can be easily scaled up or down which simplifies resource management and helps to handle different workload levels. In addition, orchestration tools like Kubernetes automate deployment, scaling and management of containerized apps.
- **Efficiency:** Since containers have less overhead compared with traditional virtual machines because they share the host system's kernel they are more efficient in deploying applications.

3. Infrastructure as Code (IaaC)

- **IaaC** is a DevOps practice where infrastructure components are managed and provisioned using machine-readable configuration files, rather than through manual processes. With this practice, the setup of infrastructure can be automated and standardized.
- **Declarative Configuration:** IaaC usually implies writing declarative code that defines the intended condition of underlying architecture (such as network settings, server configurations). These configurations are interpreted by Terraform, AWS CloudFormation and others to provision and manage infrastructure.
- **Reusability and Consistency:** By allowing configuration to be reused, IaaC provides for its version control thus ensuring uniformity in different environments. This way human errors can be minimized while eliminating deviations between environments.
- **Automated Provisioning:** IaaS scripts enable engineers to automatically provision and manage infrastructure. They make it possible to instantly create consistent environments for development, testing or production purposes
- **Scalability and Flexibility:** When demand fluctuates, IaaS allows resources on the cloud to scale up or down accordingly. It is also designed to support modifications on-the-fly so that teams can quickly adapt for new needs.

© Copyright upGrad Education Pvt. Ltd. All rights reserved

3. Version Control Systems

3.1 Version control

Version control is a system that keeps track of changes made to a file or files over time so you can recall specific versions later". It's an invaluable tool in software development which allows several developers to work together on the same project, handle code modifications while keeping track of how the project has evolved. There are two main types of version control systems: centralised and distributed ones, and Git, Subversion (SVN), and Mercurial are some popular tools.

Key Concepts of Version Control:

1. Repository
    - A repository or "repo" refers to a database that stores all files and their change histories. A repository could be either local, i.e., residing on the developer's machine or remote i.e., on a server.
2. Commit
    - Changes in repositories are often represented as commits. They describe what was changed, who made those changes, as well as when they happened. Commits act as save points for developers to go back to if necessary.
3. Branch
    - It is an independent line of development. Branches help developers work on new features, bug fixes or experiments without affecting the primary codebase. The primary branch (also known as main or master) typically represents the stable version of the code.
4. Merge:
    - A merge is the act of bringing together changes from another branch into a different one. For example, a developer may merge it to a main branch after accomplishing some new functionality in a separate one.
5. Conflict:
    - Conflict arises when changes between different branches cannot be automatically merged due to their colliding influence on same part of file. Thus, before carrying out merging, developers should manually resolve conflicts.

© Copyright upGrad Education Pvt. Ltd. All rights reserved

6. Tag:
    - Tags are marks used at specific history moments of repositories, mostly for releases (e.g., v1.0, v2.0). They provide links to specific commits.
7. Clone:
    - Cloning of repository means that an exact copy of the repository including its full history is made. Among distributed version control systems like Git this is often practiced.

Types of Version Control Systems:

1. **centralised Version Control Systems (CVCS):**
    - CVCS has got just one central repository which all members connect to; hence Subversion (SVN) becomes popular here. Despite being straightforward CVCS can develop into bottlenecks and single source of failures sometimes being so complex as well.
2. **Distributed Version Control Systems (DVCS):**
    - Every developer in DVCS owns their own local copy of the whole of the repository including all of its history. DVCS tools such as Git and Mercurial are popular. This type of configuration offers much more flexibility because it allows developers to work offline and have a full backup of the project.

Benefits of Version Control:

1. Collaboration:
    - Several developers can simultaneously work on one project without over writing each other's modifications through version control. It helps to handle and merge different contributions.
2. History and Accountability:
    - This provides a comprehensive record keeping on people who changed what part of code at which time. This openness is useful for audit purposes, as well as understanding how the project has evolved.
3. Backup and Recovery:
    - The repository's history serves as a back up. In case new changes bring bugs or issues, programmers can revert to prior versions.

© Copyright upGrad Education Pvt. Ltd. All rights reserved

4. Branching and Experimentation:
    - Developers can experiment with new functionality or fixes in branches without affecting the main codebase. This promotes a safer and more flexible development process.
5. Integration and Release Management:
    - Version control systems provide structure for managing and releasing code changes hence enabling continuous integration/delivery practices within organizations.

Popular Version Control Tools:

- Git is one of the most widely used distributed version control systems, because it is fast, flexible and has a strong branching/merging model.
- Subversion (SVN) is a centralised version control system which is easy to use and integrates well with other tools.
- Mercurial is another distributed version control system known for its performance and scalability.

3.2 Centralised Version Control Systems

Centralized Version Control System (CVCS) refers to a type of version control system where all the versioned files, as well as the historical data are stored in central server. This central repository acts as the project's authority by which all users must communicate with this server to access the codebase or contribute changes. Examples of centralized version control systems include Subversion (SVN) and Perforce.

Key Features of Centralized Version Control Systems

1. Central Repository:
    - This central repository stores all source codes' entire history including all versions. All developers' check outs and check ins are done here.
2. Check Out & Check In:

© Copyright upGrad Education Pvt. Ltd. All rights reserved

- Developers take files/directories “check out” from this central repository into their local computers to carry out working on them. After making changes they return (“check in”) them back into the repository again.

3. Version History:
    - The central repository keeps track of all changes, who made them, and when. It is useful for tracking changes, understanding project evolution, and audit.
4. User Authentication and Access Control:
    - Authentication mechanisms are usually implemented in CVCS to guarantee that only authorized users can access certain parts of the repository or perform some actions. These may include limiting reading access and writing permissions as well as branch restrictions etc.
5. Locking Mechanism:
    - Some CVCS tools offer a file locking mechanism to prevent multiple users from editing the same file simultaneously. This can be useful for binary files or files that are difficult to merge.

Advantages of centralised VCS:

1. Simplicity:
    - CVCSs are often simpler to understand and use, especially for teams with less experience in version control. The single source of truth provided by the central repository makes it clear.
2. centralised Backups:
    - Because all data is stored in one place, backing up the central repository is straightforward. This centralisation can simplify disaster recovery processes.
3. Access Control:
    - Administrators can easily control access to the repository, setting permissions at various levels, such as branch, file, or user levels. This control can enhance security and ensure that sensitive areas of the codebase are protected.
4. Conflict Resolution:
    - This centralised nature can make it easier to handle and resolve conflicts because all modifications pass through a focal point.
5. Centralised Management:

© Copyright upGrad Education Pvt. Ltd. All rights reserved

- All the data, including codes, commits and histories, are in one place. It is easier to manage because of this centralized management that also simplifies things like user permissions and repository maintenance.

Disadvantages of centralised VCS:

1. Single Point of Failure:
   - The central server represents a single point of failure where if the server goes down there will be no commits or access to the latest version of codebase leading to development's halt until. The server is restored.
2. Network Dependency:
   - Developers need to connect to the internet for them to interact with the repository. This can limit developers who work remotely or places with unreliable internet connectivity.
3. Scalability Issues:
   - The central server may become a bottleneck as teams expand and more data accumulates. There could be performance problems on large teams or large codebases, especially during peak traffic times.
4. Limited Offline Work:
   - Since any offline working needs connection back online before committing changes, developers are limited in terms of working offline since the central repository stands as an authoritative source only.

Use Cases and Considerations:

CVCS is suitable for:

- **Small to Medium-Sized Teams:** Teams that have a dependable internet connection, and do not require constant administration of a centralised server.
- **Projects with Strict Access Control Needs:** where there is need for tight control access like in regulated Industries
- **Environments Where Simplicity Is Preferred:** for individuals or groups who just want an uncomplicated version control system without the complications of distributed systems.

© Copyright upGrad Education Pvt. Ltd. All rights reserved

Centralized version Control Systems (CVCS) can still be used by organizations seeking simplicity that remains centralized based. Nevertheless, since the advent of distributed version control systems like Git, CVCS use has decreased across many open-source and large-scale enterprise projects. None the less it could still prove useful in situations where centralization management and clear workflows are beneficial.

3.3 Distributed Version Control Systems

Distributed Version Control Systems (DVCS): These are a type of version control system wherein each contributor receives their own full copy of the repository with all changes ever made to it kept intact.

Unlike CVCS, distributed VCS does not have a single central repository. Instead, each user can clone the repository, work on it separately and later disseminate his/her changes to others.

Key Features of Distributed Version Control Systems:

1. Local repositories:
    - Every developer has a local one which includes only all activities in the project. Off line working is made possible by this local database for all operations such as commits, branches or merge thus making it more flexible and faster.
2. Cloning:
    a. It helps to make a full copy of the entire repository including all branches tags and history therefore, any programmers are able to access the whole state of project at any time.
3. Branching and merging;
    a. Branching and merging are basic functions in DVCS. Creating, switching and merging branches is usually very fast hence enabling more flexible workflows and experimental development.
4. Decentralization:
    a. The lack of dependence on central servers implies that there will be no single point of failure as every user possesses an entire copy of the repository. In this scenario programmers can work independently while sharing changes with others without need for central server.

© Copyright upGrad Education Pvt. Ltd. All rights reserved

5. Pull & push
    a. Changes can also be brought into from other repositories (pull) or transmitted to other repositories (push). This makes possible a flexible mode of collaboration where programmers may decide which changes should be integrated.
6. Collaboration:
    a. Among the workflows DVCS supports are feature branching, fork and pull requests, and direct commits to shared repositories. This allows for different team structures and collaboration models.

Advantages of Distributed VCS:

1. Resilience and Redundancy:
    - Since every clone is a full backup of the repository, the system is resilient to data loss. If any single copy disappears or gets corrupted, others can retrieve it.
2. Offline Work:
    - Developers can do most version control operations (commits, diffs, history exploration) even without Internet connection. This is useful in remote work or where there is bad internet access.
3. Scalability:
    - DVCS scales more easily to larger projects and teams because things like branching and merging happen locally which typically makes them faster.
4. Flexibility in Workflows:
    - Teams may choose different workflows like Gitflow, GitHub Flow or custom workflows depending on their requirements. This flexibility accommodates variations in development processes and project expectations.
5. Efficient Collaboration:
    - DVCS has support for multiple collaboration models such as centralised workflows (with a single main repository) and decentralised workflows (peer-to-peer).
6. Improved Performance through:
    - Operations such as viewing history, branching out, and committing are usually faster because they are done locally.

© Copyright upGrad Education Pvt. Ltd. All rights reserved

Drawbacks of Distributed VCS:

1. Complexity:
    - DVCS can be more difficult to setup and manage especially for those who have never used version control systems before. Additional concepts like local vs remote repositories make things even harder.
2. Initial Cloning Time:
    - Cloning a large repository may take time and require significant disk space as the whole history is downloaded.
3. Conflict Resolution:
    - With many independent repositories, conflicts will happen more often and will need careful management and resolution strategies.

Popular Distributed VCS Tools

1. Git
    - The most common DVCS which is famous for its speed, flexibility and robustness. It is Git that forms the basis of numerous well-known platforms like GitHub, GitLab or Bitbucket.
2. Mercurial
    - Another DVCS that stands out due to its simplicity and performance. Mercurial is employed in projects such as the Python programming language or Mozilla.

Use Cases and Considerations

DVCS is best suited for:

- **Large Teams and Open Source Projects:** Where developers are spread across different locations yet required to work independently on their own codes.
- **Complex Projects:** That involve extensive branching-out operations plus merging.
- **Organisations That Value Redundancy and Resilience:** A situation where it is advantageous to have a backup of the repository on each developer's machine.

Distributed Version Control Systems (DVCSs) are increasingly preferred by many organizations and projects mainly for their adaptability, resilience and supporting

© Copyright upGrad Education Pvt. Ltd. All rights reserved

collaborative work flows. They offer more dynamic and efficient development practices, making them well-suited for contemporary software development.

3.4 Git and various commands

Git is a distributed version control system commonly used for tracking code changes in software development. It allows multiple developers to contribute simultaneously, tracks versions and encourages collaboration among team members. Developed by Linus Torvalds in 2005, Git has since been considered as the standard version control system due to its performance, flexibility, and dependability.

Key Concepts in Git

1. **Repository (Repo):**
    - A git repository is a folder that contains all project files with the full history of changes. This can be located either locally (on a developer's computer) or remotely (on server or web services such as GitHub).
2. **Commit:**
    - A commit is a snapshot of the changes made to the files in the repository. It contains a unique ID, with it comes a message describing these, and metadata such as authorship which includes timestamp.
3. **Branch:**
    - A branch is a separate line of development. The main branch in most Git repositories is usually called 'main' or 'master', while others are used for developing features, bug-fixing etc.
4. **Merge:**
    - Merging is the process through which we can integrate changes from one branch into another. For instance, if there are no conflicts, Git will automatically merge changes; however if there are conflicting changes manual intervention is required.
5. **Pull and Push:**
    - Pull brings and combines/ merges (changes) from remote repository to local repository. In contrast, push transmits local alterations to a remote repository.
6. **Clone:**

© Copyright upGrad Education Pvt. Ltd. All rights reserved

- Cloning creates an entire replica of a remote repository including all branches and history on our local machine.

Common Git Commands

Setup and Configuration

- `git init`:
  - Initialises a new Git repository in the current directory.
- `git clone <repository_url>`:
  - Clones an existing remote repository to the local machine.
- `git config --global user.name "Your Name"`:
  - Sets the global username for Git commits.
- `git config --global user.email "your.email@example.com"`:
  - Sets the global email address for Git commits.

Working with Changes

- `git status`:
  - Shows the working directory status, including staged, unstaged, and untracked files.
- `git add <file>`:
  - Stages a file for the next commit.
- `git add .`:
  - Stages all changes (new files, modifications, deletions) for the next commit.
- `git commit -m "Commit message"`:
  - Commits staged changes with a message describing the changes.
- `git diff`:
  - Shows changes between the working directory and the index (staging area).

Branching and Merging

- `git branch`:
  - Lists all local branches in the repository.
- `git branch <branch_name>`:
  - Creates a new branch.
- `git checkout <branch_name>`:
  - Switches to the specified branch.
- `git checkout -b <branch_name>`:
  - Creates and switches to a new branch.
- `git merge <branch_name>`:
  - Merges the specified branch into the current branch.

© Copyright upGrad Education Pvt. Ltd. All rights reserved

Synchronising with Remote Repositories

- `git remote add origin <repository_url>`:
  - Adds a remote repository URL with the alias "origin."
- `git fetch`:
  - Downloads objects and refs from another repository.
- `git pull`:
  - Fetches and merges changes from the remote repository into the current branch.
- `git push`:
  - Pushes local changes to the remote repository.

Viewing History and Logs

- `git log`:
  - Shows the commit history for the current branch.
- `git log --oneline`:
  - Displays the commit history in a concise, single-line format.
- `git show <commit_hash>`:
  - Shows the changes introduced by the specified commit.

Undoing Changes

- `git reset <file>`:
  - Unstages a file, keeping the changes in the working directory.
- `git reset --hard <commit_hash>`:
  - Resets the repository to the specified commit, discarding all changes.
- `git revert <commit_hash>`:
  - Creates a new commit that undoes the changes made by a specific commit.

Tagging

- `git tag <tag_name>`:
  - Creates a tag at the current commit.
- `git tag -a <tag_name> -m "Tag message"`:
  - Creates an annotated tag with a message.

Advanced Features

- **Rebase**: (`git rebase <branch>`) Reapplies commits on top of another base tip. Used for linearising commit history.
- **Cherry-pick**: (`git cherry-pick <commit_hash>`) Applies changes from specific commits to another branch.

© Copyright upGrad Education Pvt. Ltd. All rights reserved

- **Stashing**: (`git stash`, `git stash apply`) Temporarily shelves changes made to the working directory, useful for switching branches without committing incomplete work.

Git's extensive feature set, combined with its distributed nature, makes it a powerful tool for managing projects of all sizes. It supports complex workflows and is an essential part of modern software development practices.

3.5 GitHub

GitHub is a web-based platform that hosts version control through Git. The platform is equipped with numerous features that enhance cooperation in software development making it one of the leading platforms for sharing codes and managing projects. It was established in 2008 and has now become a central point for open source projects as well as private repositories for single developers and organizations.

Main Features of GitHub

1. **Repositories:**
    - These are stores where project files are kept together with full history about their revisions on GitHub. Public as well as private ones can be created by users. While anyone can access public repositories, private repositories are limited to certain users.
2. **Forking:**
    - Forking refers to the process of creating an individual's copy of another person's repository. This is useful if you want to make some independent changes or experiments without affecting the original repository. This happens frequently in open-source projects where users may wish to contribute.
3. **Pull Requests:**
    - This feature allows developers to inform others about any changes made on a repository using pull requests (PRs). Maintainers can review these changes, discuss potential adjustments, and then merge them into the base code if they deem it appropriate.
4. **Issues and Issue Tracking:**

© Copyright upGrad Education Pvt. Ltd. All rights reserved

- For every repository, GitHub has an integrated issue tracker that allows users to report bugs, suggest features, and track progress. Issues can be labeled, assigned to team members, and linked with particular pull requests or commits.

5. **GitHub Actions:**
   - GitHub Actions is a CI/CD (Continuous Integration/Continuous Deployment) tool that lets you automate workflows for building, testing, and deploying code. It works by using YAML files for defining workflows which makes it easier to integrate with the development process.
6. **GitHub Pages:**
   - With GitHub Pages, people can host static websites straight from their GitHub repositories. Most commonly used for project documentation, personal blogs as well as landing pages of open source projects.
7. **Collaborators and Teams:**
   - GitHub enables repository owners to add collaborators and manage teams. This feature helps organise and manage access permissions making it easier for organisations to control who can read, write and administer repositories.
8. **Wiki:**
   - A wiki is a collection of pages that can be used to document the project; each GitHub repository may have one such associated with it. Wikis are editable by repository collaborators providing a handy location for storing project documentation.

- **Social Features:**
  - GitHub has social networking capabilities such as following other users, starring repositories, and watching projects. These features allow developers to explore exciting projects and keep up to date with their developments.
- **Security Features:**
  - GitHub also has many security features such as dependency vulnerability alerts, code scanning, secret scanning, and two-factor authentication. These features ensure that codebases are secure and remain uncorrupted.
- **Project Boards:**

© Copyright upGrad Education Pvt. Ltd. All rights reserved

- GitHub also has project boards resembling Kanban boards that help you sort out and prioritize work. With these boards, issues, pull requests and notes can be managed visually and interactively.

Use Cases

- **Open Source Projects:** GitHub is the main platform for hosting open-source projects thus making it easy for developers around the world to collaborate, contribute and share code with one another.
- **Enterprise Development:** Companies use GitHub as a private repository for internal collaboration and integrating DevOps workflows.
- **Education:** GitHub is used widely in schools/colleges managing course materials assignments or coding projects.
- **Personal Projects:** Individual developers use GitHub to host personal projects or portfolios as well as experiment with new technologies.

Integrations and Ecosystem

Slack, Trello, Jira and continuous integration services such as Travis CI and CircleCI are examples of some of the third-party tools and services that can be integrated with GitHub. As a result, it is an infinitely flexible platform for developers who employ multiple instruments in their workflow.

GitHub Enterprise

For larger organisations that have specific security and compliance needs, GitHub provides an option called **GitHub enterprise** which is either self-hosted or cloud-based version of GitHub having extra features and controls.

GitHub has become an important part the modern software development ecosystem. It boasts robust features and a community-driven approach that makes it an invaluable resource for developers, teams as well as organizations who want to effectively manage their code bases and collaborate on projects.

© Copyright upGrad Education Pvt. Ltd. All rights reserved

4. Introduction to Docker

4.1 Docker

The open-source system helps automate application deployment, scaling, and management through containerization. Containers bundle together apps together with all their dependencies hence enabling it to run consistently across different environments. Docker simplifies workflows thus making them faster by availing tools together with a platform to create, ship out and deploy applications in containers.

Key Concepts in Docker

1. **Containers:**
    - Containers are light, movable, and independent software components containing all the necessary to run an app such as application code, runtime, libraries and system tools. While they share host operating system kernel, they have isolation from each other hence providing uniform runtime environment for different systems.
2. **Images:**
    - Docker image is a self-contained, executable package of software that includes everything required to run a piece of software including the code, runtime, libraries and configurations. Images are immutable files used in the creation of docker containers.
3. **Dockerfile:**
    - A Dockerfile is a textual representation filed with instructions on how to build an image on a Docker platform. It contains the base image name as well as defines what should be packaged within the images such as main application codes, ENV variables among others.
4. **Docker Hub:**
    - Docker Hub is an online registry service where you can find and store images for Docker. It acts as a central hub for container images while enabling developers to access public or private repositories of their own choice.
5. **Docker Engine:**

© Copyright upGrad Education Pvt. Ltd. All rights reserved

- This is core software responsible for building and running docker containers which can be installed on various operating systems including Linux OS versions 7 and above up to Windows Server 2016.

6. **Volumes**:
   - Volumes are used to persist data generated and used by Docker containers. They enable data to be shared between containers and the host system, providing a way to store data that should not be part of the image.
7. **Networking**:
   - Docker provides various networking options for containers, including bridge networks, host networks, and overlay networks. These options allow containers to communicate with each other and with the host system.

Key Docker Commands

1. **docker run**:
   - Creates and starts a new container from a specified image.
   - Example: `docker run -it ubuntu bash` (starts a container with an interactive terminal)
2. **docker build**:
   - Builds a Docker image from a Dockerfile and a context (a set of files and directories).
   - Example: `docker build -t myapp .` (builds an image with the tag "myapp")
3. **docker pull**:
   - Downloads an image from a Docker registry, such as Docker Hub.
   - Example: `docker pull nginx` (pulls the latest Nginx image)
4. **docker push**:
   - Uploads an image to a Docker registry.
   - Example: `docker push username/myapp` (pushes the image "myapp" to the user's repository)
5. **docker ps**:
   - Lists all running containers.
   - Example: `docker ps` (shows details of running containers)
6. **docker stop**:
   - Stops a running container.
   - Example: `docker stop container_id` (stops the container with the specified ID)
7. **docker rm**:
   - Removes a stopped container.
   - Example: `docker rm container_id` (removes the specified container)
8. **docker rmi**:
   - Removes an image from the local system.
   - Example: `docker rmi image_id` (removes the specified image)

© Copyright upGrad Education Pvt. Ltd. All rights reserved

9. **docker exec**:
    - Runs a command in a running container.
    - Example: `docker exec -it container_id bash` (opens a bash shell in the specified container)
10. **docker-compose**:
    - A tool for defining and running multi-container Docker applications. It uses a YAML file to configure the application's services, networks, and volumes.
    - Example: `docker-compose up` (starts the services defined in the docker-compose.yml file)

Benefits of Docker

1. **Consistency:**
    - Containers in Docker guarantee that applications run the same way irrespective of where they are deployed. Consequently, this consistency eliminates “it works on my machine” problem.
2. **Isolation:**
    - The containers provide segregated environments for application execution. These isolate the conflicts among different applications running on a single host.
3. **Portability:**
    - Docker containers can be executed on any system with Docker capacity, including local machines, on-premises servers and cloud infrastructure. This portability makes it easy to move applications between environments.
4. **Efficiency:**
    - Containers share the host system’s kernel and resources thereby making them more lightweight and efficient compared to traditional virtual machines.
5. **Scalability:**
    - Docker is light enough to horizontally scale applications through multiple container instances.

- **Ease of Deployment:**
    - By collapsing the application together with its dependencies, docker simplifies deployment processes. This packaging thus helps in better dependency management and version control for production deployments

Use Cases

© Copyright upGrad Education Pvt. Ltd. All rights reserved

- **Microservices Architecture:** For microservices whose each service may run in a separate container, docker proves ideal for deploying such services.
- **DevOps and CI/CD:** Docker has found common use in continuous integration and continuous deployment pipelines to automate testing and deployment processes.
- **Development Environments**: Developers use Docker to create consistent development environments that mirror production.
- **Cloud and Hybrid Deployments**: Docker's portability makes it suitable for deploying applications across different cloud providers and on-premises environments.

Docker has revolutionized how software is developed, shipped, and run. Its containerization technology provides a lightweight, efficient, and consistent environment for applications, making it an essential tool in modern software development and deployment practices.

4.2 Docker v/s Virtual Machines

Docker and **Virtual Machines (VMs)** are both technologies used to create isolated environments for running applications. However, they operate differently and have distinct advantages and use cases. Here's a comparison of Docker containers and virtual machines:

Feature	Docker Containers	Virtual Machines (VMs)
Architecture	Virtualize at the OS level	Virtualize at the hardware level
	Share the host OS kernel	Includes a full guest OS
	Isolated processes	Managed by a hypervisor
Components	Application, dependencies, system tools	Full guest OS, hypervisor, virtualized hardware
Resource Utilization	Lightweight, lower overhead	More resource-intensive, higher overhead
	Efficient use of CPU, memory, and storage	Requires more memory, CPU, and storage
Isolation	Process-level isolation	Stronger isolation with full OS
Performance	Near-native performance	Slightly slower due to virtualization overhead

© Copyright upGrad Education Pvt. Ltd. All rights reserved

	Quick startup times (seconds)	Longer boot times (minutes)
Portability	Highly portable across environments	Less portable, dependent on a virtualization platform
Use Cases	Microservices, DevOps, CI/CD, cloud-native apps	Multi-OS environments, legacy applications, strong isolation needs
Management	Managed via Docker Compose, Kubernetes, etc.	Managed via hypervisors, virtualization platforms
Scalability	Easily scalable, rapid adjustments	More complex scaling, requires resource allocation
Examples of Tools	Docker, Docker Compose, Kubernetes	VMware vSphere, Microsoft Hyper-V, VirtualBox

4.3 Docker Architecture

The **Docker architecture** consists of several key components that work together to provide a complete environment for developing, shipping, and running applications in containers. These components include the Docker client, Docker daemon, images, containers, and a registry. Here's an overview of each component and how they interact within the Docker architecture:

1. **Docker Client**

The Docker client is a command-line interface (CLI) tool that allows users to interact with the Docker daemon. It is the primary way users manage Docker containers, images, and other resources. The Docker client sends commands to the Docker daemon, which then carries out the requested actions.

- **Commands**: The Docker client provides commands such as `docker run`, `docker build`, `docker pull`, `docker push`, and more, which are used to manage containers, images, and other Docker objects.

2. **Docker Daemon (dockerd)**

Background-running daemon docker daemon, commonly called dockerd, is a process running on the host machine waiting for commands from the Docker client and managing Docker objects

© Copyright upGrad Education Pvt. Ltd. All rights reserved

like images, containers, networks and volumes.The Docker daemon also manages other daemons that run Docker services.

Responsibilities: It can be used to build, run and deploy containers in different servers. It can manage multiple containers and handle life cycle of each container.

3. Docker Images

Docker images are templates that are read-only which are used in creation of containers. An image contains everything needed to run an application – the code itself, runtime, libraries, environment variables and configuration files. Images are created as series of layers with each layer representing different changes made.

- **Building Images:** The construction of Docker images relies on a Dockerfile that incorporates a set of instructions for assembling the image. After reading the Dockerfile and creating a new layer for each instruction, the Docker daemon builds the image.
- **Layering:** Each level in an image is a modification like adding a file or installing a package .Layers are cached and only need to be rebuilt if the layers change.

4. Docker Containers

Docker containers are Docker images that run on the host as isolated processes. They wrap up an app together with its required libraries to produce the same results regardless of the platform.

Lifecycle: Containers can be created, started, stopped, paused, and deleted. They are not permanent, which means changes made in a running container are lost unless they're saved in a Docker volume or committed into another image.

Isolation: Containers isolate each other and the host system by using namespaces and control groups (cgroups). This ensures that processes running within one container don't interfere with others or with the actual host itself.

© Copyright upGrad Education Pvt. Ltd. All rights reserved

5. **Docker Registry**

A Docker registry is where Docker stores its images. It may be public or private. One of such well-known public registries is Docker Hub but organizations might also have their private registries.

- **Docker Hub:** This is one of many publicly accessible registries hosting thousands of Docker images including official ones for popular applications and OS distributions.
- **Private Registries:** Proprietary pictures can be stored by organizations in private registries for security purposes.
- **Interaction:** Users send images to be saved on a registry and pull them from a registry to run on their system.

6. **Additional Components**
    - **Docker Compose:** Docker Compose is a tool that lets users define and manage multi-container Docker applications with the aid of a docker-compose.yml file. It provides an easy way to handle containers, networks and volumes.
    - **Docker Network:** Docker has its own network layer which allows containers to communicate with each other as well as the outside world. Docker networks can be customized, including bridge networks, host networks, and overlay networks.
    - **Docker Volumes:** These are used by Docker containers to persist data generated within them. They are storage systems that exist beyond container instances thus can be used for passing information amid other container instances or even between the hosts' file system.

Docker Workflow

1. **Build:** A user writes a Dockerfile specifying the environment settings and dependencies for the application being developed after which the Docker daemon creates an image from this Dockerfile.
2. **Ship:** The created image is then pushed out to a Docker registry such as dockerhub where it could be deposited with other people.

© Copyright upGrad Education Pvt. Ltd. All rights reserved

3. **Run:** Users may pull the image from the registry and run it on any Docker-enabled host as a container. The container then runs the application in an environment that is consistent and isolated.

4.4 Docker Image

A **Docker image** is an executable package of software that is lightweight and standalone. It contains everything required to run a specific app such as runtime, application code, libraries, environment variables, configuration files etc., and all other dependencies needed by the application to execute. Docker images are templates for creating Docker containers.

Key Characteristics of Docker Images:

1. Immutability:
    - Docker images are immutable; once they are made they cannot be changed but when updates are necessary new images should be built.
2. Layered File System:
    - Docker images consist of layers which each represent a set of changes such as adding a file or installing some packages among others. These layers are stacked one above another to make up the final image whereby every layer is read-only apart from the topmost layer that is writable during execution of a container.
3. Shared Layers:
    - If layers are same, they are shared between images. This attribute helps save disk space and minimizes the time for transferring images over a network as the unchanged layers do not need to be downloaded again.
4. Base Images:
    - A Docker image is created based on a base image. It could be an operating system (like Ubuntu, Alpine), a runtime environment (like Node.js, Python), or a minimal setup. A base image does not have a parent image.
5. Tags:

© Copyright upGrad Education Pvt. Ltd. All rights reserved

- The tag and name of the images distinguish them from one another. The default tag is latest but other tags that are used to indicate different versions of an image such as nginx:1.19, nginx:1.20 can be specified by the user.

Building Docker Images

There's usually a Dockerfile when building docker images which is basically the script of instructions followed by Docker in making up an image. To define an image's content and configuration, a Dockerfile includes commands like FROM, RUN, COPY and EXPOSE.

Key Dockerfile Instructions:

- **FROM**: Specifies the base image to use. This is the first instruction in a Dockerfile.
- **RUN**: Executes commands in a new layer and commits the results. Used for installing software packages or other setup tasks.
- **COPY**: Copies files from the host machine into the Docker image.
- **WORKDIR**: Sets the working directory inside the container.
- **CMD**: Specifies the default command to run when the container starts. Only one `CMD` instruction is allowed, and it can be overridden at runtime.

Using Docker Images

1. **Pulling Images**:
   - Images can be pulled from a Docker registry like Docker Hub using the command `docker pull <image_name>`. For example, `docker pull nginx` pulls the latest Nginx image.
2. **Running Containers**:
   - Containers are instances of Docker images. To run a container from an image, the command `docker run <image_name>` is used. For example, `docker run nginx` starts a container using the Nginx image.
3. **Listing Images**:
   - The command `docker images` lists all images available on the local system.
4. **Tagging and Pushing Images**:
   - Images can be tagged using `docker tag <image_id> <repository>/<image_name>:<tag>`. Tagged images can be pushed to a Docker registry with `docker push <repository>/<image_name>:<tag>`.

Managing Docker Images

© Copyright upGrad Education Pvt. Ltd. All rights reserved

- **Image Layers:** Docker employs a union file system in managing image layers, which allows for efficient storage and transfer of images. Shared layers among images help to reduce redundancy.
- **Image Size:** A good practice is to ensure that the image size is kept as low as possible by reducing the number of layers and including only what is necessary. By doing this optimization, it will be possible to deploy in less time and resources.

4.5 Docker Registry

A **Docker registry** is a storage and distribution system for docker images, which stores Docker images and enables users to share, distribute, and manage these images. It plays a crucial role in the overall functionality of Docker by allowing an efficient and secure distribution of docker images across different environments or organizations.

Key Concepts of Docker Registry

1. **Repositories**:
    - A repository in a Docker registry is a collection of related Docker images. Each repository can contain multiple versions of an image, identified by tags. For example, a repository for an application called "myapp" might contain images tagged as `latest`, `1.0`, `2.0`, etc.
2. **Tags**:
    - Tags are labels attached to Docker images to identify specific versions. They provide a way to differentiate between different builds of an image in a repository. For example, `nginx:1.19` and `nginx:1.20` refer to different versions of the Nginx image.
3. **Registry Types:**
    - **Public Registries:** These are open to the public and can be used by anyone; Docker Hub being widely used public registry.
    - **Private Registries:** These can only be accessed by certain people or group of people as they offer higher levels of security making them ideal for storing proprietary information photosimages etc… Utilizing private registries allow more control over accessibility and security matters.

Key Features and Functions of Docker Registry

1. **Image Storage:**

© Copyright upGrad Education Pvt. Ltd. All rights reserved

- Docker registries hold all the layers that make up an image. The images are uploaded (pushed) into a registry while they can also be downloaded (pulled) from there.

2. **Image Distribution:**
   - For instance, registries help in disseminating Docker images to different environments like testing, development and production. Consequently this makes it possible for deployment to be done consistently.
3. **Access Control:**
   - In particular, access controls mechanisms are often implemented in registries especially if they are private ones. This involves authenticating the user, controlling access based on roles and identities being managed by identity management systems.
4. **Image Management**
   - Conversely, images can be managed using tools and APIs offered by registries such as listing repositories, retrieving metadata for an image or deleting old or unused images.
5. **Security Scanning**
   - Some registries have security scanning functionality whereby images can be checked for vulnerabilities. It is helpful when it comes to identification and handling of any potential security problems that may arise before a release.

Popular Docker Registries

1. **Docker Hub:**
   - This image acts as a default public registry provided by Docker. It has many images including official ones for popular software applications. Both public and private repositories exist within Docker Hub which also hosts automated builds alongside organizational accounts.
2. **Amazon Elastic Container Registry (ECR):**
   - Amazon ECR is one of the fully managed Docker registry services provided by AWS. It integrates with other Amazon Web Services while offering options of having vulnerability scans as well as image lifecycle policies in place among others

© Copyright upGrad Education Pvt. Ltd. All rights reserved

3. **Google Container Registry (GCR):**
    - GCR is a docker registry service provided by Google Cloud Platform. It works with Google Kubernetes Engine (GKE) and provides secure, scalable storage for images.
4. **Azure Container Registry (ACR):**
    - Azure Container Registry (ACR) is the Azure offering for a managed Docker registry that supports all Docker CLI commands and interacts well with Azure Kubernetes Service (AKS).
5. **Harbor:**
    - It is an open-source container registry which comes with advanced features such as security scanning, content signing, and identity management. This can be used in an enterprise to manage container images.
6. **Artifactory:**
    - JFrog Artifactory is a universal artifact repository manager that supports –among others- Docker images. It offers features like high availability, disaster recovery, and hybrid environments for the enterprise.

Basic Commands for Interacting with Docker Registries

- **docker pull <repository>/<image>:<tag>**:
    - Downloads an image from a registry to the local machine. If the tag is omitted, the `latest` tag is assumed.
    - Example: `docker pull nginx:1.20`
- **docker push <repository>/<image>:<tag>**:
    - Uploads an image from the local machine to a registry.
    - Example: `docker push myrepo/myapp:2.0`
- **docker login**:
    - Logs in to a Docker registry. This is required for pushing images to private registries and can also be used for authenticated pulls.
    - Example: `docker login -u username -p password myregistry.com`
- **docker tag <image_id> <repository>/<image>:<tag>**:
    - Tags an image with a new name and tag. This is often done before pushing an image to a registry.
    - Example: `docker tag abc123 myrepo/myapp:2.0`

Use Cases and Best Practices

1. Continuous Integration/Continuous Deployment (CI/CD):

© Copyright upGrad Education Pvt. Ltd. All rights reserved

- Docker registries play an important role in CI/CD pipeline where they hold images for different stages of the pipeline that include testing, staging, and production.

2. Microservices Architecture:
    - In microservices architecture, one can package each service as their own separate docker image ready to be stored in a registry; thereby allowing uniform deployment and scaling of services.
3. Security and Compliance
    - Implementing security scans and access controls can help maintain compliance with organizational policies and regulations. In addition, private registries provide a secure environment for storing and distributing proprietary software.

4.6 Docker containers

4. These are portable, lightweight, and autonomous units that marriage an app with its supporting software components to ensure uniformity of operation in any computing environment. Containers are an integral part of the Docker platform for the delivery, development and deployment of applications.

Key Features of Docker Containers

1. Isolation:
    - In this way, each container runs independently without interfering with others through process and file system isolation using Linux kernel features such as namespaces and control groups (cgroups).
2. Lightweight:
    - Instead of having the whole OS on a virtual machine, containers share only the application along with its dependencies on the host system's kernel reducing their weight significantly compared to VMs hence requiring less overhead due to faster startup time.
3. Portability:
    - Therefore, they can be run uniformly across diverse environments—developer laptops, on-premises data centers or cloud deployments—since Docker containers bundle together an application plus its dependencies.
4. Immutable

© Copyright upGrad Education Pvt. Ltd. All rights reserved

- Docker containers are based on Docker images, which are immutable. When an image is created, it cannot be modified. To update the application, a new image is built and then a new container is started from that image.

5. Transient:
    - Containers are designed to be stateless and transient. Data that needs to persist beyond the life of the container typically resides in Docker volumes or external storage systems.
6. Efficient use of resources:
    - Containers are more resource efficient compared to traditional virtual machines as they share the underlying Operating System Kernel and can efficiently use CPU, memory, and storage.

How Docker Containers Work

1. Docker Image:
    - A Docker image is a read-only blueprint that contains the application code, libraries, environment variables, configuration files and other dependencies required to run the application. An image is used to create one or many containers.
2. Creating a Container:
    - A container is built from a Docker image. As part of this process, Docker adds a writable layer on top of its set of read-only layers which enables changes to file system by the container itself.
3. Container Execution:
    - Containers can be started after their creation. While this occurs, a container runs the commands specified in the Dockerfile or passed at runtime. As such, it is possible to start, pause and stop containers.
4. Networking:
    - Docker networking allows communication between Docker containers and other systems outside of them. Consequently, virtual networks permit secure, efficient communication between containers.
5. Volumes:

© Copyright upGrad Education Pvt. Ltd. All rights reserved

- Docker volumes are able to persist any data that is accumulated by Docker containers. Moreover, these volumes do not die when a container dies i.e. data stored in them will be there even after a container has been deleted.

Managing Docker Containers

Docker provides a command-line interface (CLI) for managing containers. Some common commands include:

- `docker run`:
  - Creates and starts a new container from a specified image.
  - Example: `docker run -d --name myapp nginx` (starts a container named "myapp" using the Nginx image in detached mode)
- `docker ps`:
  - Lists all running containers.
  - Example: `docker ps` (shows details of running containers)
- `docker stop`:
  - Stops a running container.
  - Example: `docker stop myapp` (stops the container named "myapp")
- `docker rm`:
  - Removes a stopped container.
  - Example: `docker rm myapp` (removes the container named "myapp")
- `docker exec`:
  - Runs a command in a running container.
  - Example: `docker exec -it myapp bash` (opens an interactive bash shell in the "myapp" container)
- `docker logs`:
  - Fetches the logs of a container.
  - Example: `docker logs myapp` (shows the logs for the "myapp" container)

Use Cases for Docker Containers

1. Microservices Architecture:
   - All microservices can run on separate containers hence they should be deployed in this way to achieve this.The benefit here is that updates and scaling of each individual micro-service can easily be done without interruption
2. Continuous Integration/Continuous Deployment (CI/CD):
   - In CI/CD pipelines, containers are frequently used so as to provide consistent environments for building, testing, and deploying applications; thus avoiding “it works on my machine” kind of problems.

© Copyright upGrad Education Pvt. Ltd. All rights reserved

3. Development Environments:
    - Developers use containers to derive reproducible development environments, which will imitate the production. This is essential in that it ensures uniformity of applications across different stages of the software lifecycle.
4. Scalable Applications:
    - Containers are fistly scalable and secondly, they can be scaled up or down to meet varying workload requirements by developers.
5. Legacy Application Modernization:
    - This is achieved through packaging and deploying legacy systems using containers hence making them more manageable as well as ready to scale while integrating them with modern systems.

Benefits of Docker Containers

- **Consistency:** Containers ensure that applications behave the same way in development, testing and production environments.
- **Isolation:** Containers contain applications and their dependencies separately avoiding conflicts resulting into a secure environment.
- **Efficiency:** Considering that containers are much lighter than traditional virtual machines, they are able to utilize resources extensively thereby enhancing efficiency in their operation.
- **Scalability:** Containers can be easily scaled thus allowing applications handle increased traffic and workloads gracefully.

4.7 Docker volume

Docker volumes helps for persistence data generated and consumed by Docker container. Unlike ephemeral nature of container file systems, volumes provide a storage mechanism for data that live beyond one instance lifecycle i.e., a definition applied when managing persistent.

Docker Volumes : Key Characteristics

1. Persistence:

© Copyright upGrad Education Pvt. Ltd. All rights reserved

- When a volume contains data it persists after the container has been removed or stopped. Thus, volumes are convenient for saving such persistent data as log files, database files and user uploads.

2. Container Isolation:
    - Volumes are controlled by Docker independently of containers thus can be used by several other containers. This isolates the storage of data hence creating an abstraction and flexibility in managing data.
3. Performance:
    - Volumes have been optimized to deliver the best performance. They can be stored either on host file system or remote storage solutions, which provides some level of flexibility when accessing or managing data.
4. Backup and Migration:
    - Since volumes are separate from containers, they can be easily backed up, restored, and migrated across different environments or hosts.
5. Access Control:
    - To restrict how containers interact with and change the information in them, specific permissions may be set for each volume.

Types of Docker Volumes

1. Named Volumes:
    - Named volumes are specifically created and managed by Docker; they are given a name that distinguishes them and can be referred in many other containers quickly.
2. Anonymous books:
    - When a container starts, anonymous volumes are created when no specific volume is defined. Being used primarily for temporary storage, these volumes do not have a name.
3. Host Volumes (Bind Mounts):
    - This can be done by using host volumes which maps a file or directory on the host machine's file system to a path in the container. Host volumes can be useful if you need to access your files or directories that are located on the host system,

© Copyright upGrad Education Pvt. Ltd. All rights reserved

but it also makes the container dependent on the structure of the file system of its host.

Creating and Using Docker Volumes

Creating a Named Volume

You can create a named volume using the Docker CLI:

```
docker volume create my_volume
```

This command creates a volume named `my_volume`.

Inspecting a Volume

You can inspect a volume to see its details, including its mount point on the host:

```
docker volume inspect my_volume
```

Using a Volume in a Container

You can attach a volume to a container using the `-v` or `--mount` flag:

```
docker run -d --name my_container -v my_volume:/data my_image
```

In this example, the volume `my_volume` is mounted to the `/data` directory inside the container.

Using a Host Volume (Bind Mount)

To use a host volume, specify the absolute path of the directory or file on the host machine:

```
docker run -d --name my_container -v /host/path:/container/path my_image
```

This command mounts the host directory `/host/path` to `/container/path` in the container.

Listing Volumes

You can list all Docker volumes on your system:

```
docker volume ls
```

Removing a Volume

© Copyright upGrad Education Pvt. Ltd. All rights reserved

To remove a Docker volume, use the following command:

```
docker volume rm my_volume
```

Volumes not associated with any containers can be removed using the `docker volume prune` command, which cleans up unused volumes.

Use Cases for Docker Volumes

1. Database Storage:
    - Typically, database files are stored in volumes so that even if the database container is stopped or re-created, data is not lost.
2. Persistent Application Data:
    - Any application which generates or consumes data required across restarts of containers can use this functionality.
3. Sharing Data Between Containers:
    - In some instances, multiple containers may share a common volume thus allowing them to exchange data. For instance, both web server and backend service could rely on one single volume for logging purposes.
4. Configuration Management:
    - Also, through employing “volumes,” configuration files and other vital artifacts can be managed thereby enabling dynamically updating without rebuilding containers.

Best Practices

- Data segregation: Ensure that persistent data is isolated from container images. Thus, when updating the containers or uninstalling them you do not lose your data.
- Disaster Recovery and Backup: To prevent loss of information, volumes should be regularly backed up. You may want to implement automated backup and restore procedures for important information.
- Access Control: Set proper authorizations to your volumes specifically if the same volume is used by multiple containers which will prevent unauthorized access.

© Copyright upGrad Education Pvt. Ltd. All rights reserved